

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY



Data Integration and Processing

Topic: Developing a Data Integration System for Managing Online Book Information

Advisor: Dr. Vu Tuyet Trinh

Group 5

Tuong-Tu, Huu	20232282M
Tien-Tung, Bui	20231058M
Trong-Bang, Nguyen	20232210M

Hanoi, August 2024

I. Introduction

In today's digital age, the availability and accessibility of book data from various sources have grown exponentially. Libraries, bookstores, online platforms, and publishers all generate massive amounts of data related to books, including metadata such as titles, authors, publication dates, ISBNs, and reviews. However, this data is often scattered across different formats, databases, and systems, making it challenging to access, analyze, and utilize effectively.

The Book Data Integration project aims to address these challenges by developing a comprehensive framework that integrates book data from multiple sources into a unified system. This project will focus on collecting, cleaning, and harmonizing data to ensure consistency and accuracy. By leveraging advanced data integration techniques and tools, the project seeks to create a centralized repository that can be used for various applications, such as recommendation systems, market analysis, and academic research. In addition, we aim to enhance the user experience by creating a book web search interface that allows users to easily search for books and access relevant information. This interface will be designed to facilitate quick and efficient retrieval of book data, enabling users to discover new titles and explore different genres seamlessly. Furthermore, the project will provide visualizations, such as trend lines, to highlight key patterns and insights within the book data, offering valuable perspectives on market trends, popular genres, and emerging literary themes. Ultimately, this project aims to make book data more accessible, usable, and valuable for all stakeholders, transforming the way book inventories and searches are managed and utilized in the digital age.

In this topic, our group conducted research and analysis of related technologies and documents. Specifically, each member performed the following tasks:

Task Assignment

- Tien-Tung, Bui: Build an Airflow automation to crawl data and save to Datawarehouse.
- Tuong-Tu, Huu: Integrate book data from different sources and clean the data.
- Trong-Bang, Nguyen: Build a book web search and visualize the data.

II. System overview

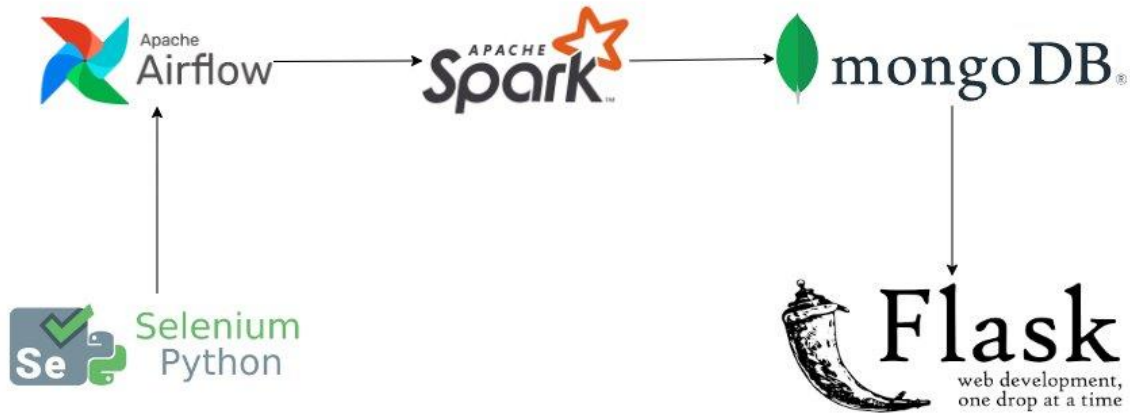


Figure 1: System overview

The system is designed as an integrated processing chain, meticulously structured to ensure that data collection, processing, and application updates are entirely automated and continuously monitored. This system comprises five primary modules, each responsible for distinct tasks and utilizing a specialized tech stack to fulfill its role efficiently:

- **Data Crawler:** This module is responsible for collecting and extracting data to produce structured datasets. It leverages BeautifulSoup for parsing HTML and XML documents and Selenium for interacting with dynamic web pages, enabling the retrieval of data from complex, JavaScript-driven websites.
- **Scheduler:** To automate the entire process of data crawling and subsequent updates, the Scheduler module is implemented using Apache Airflow. Airflow orchestrates and schedules tasks, ensuring that data collection and updates occur in a timely, repeatable manner, with full logging and monitoring capabilities.
- **Data Processor:** Once data is collected, it undergoes cleaning and integration in the Data Processor module. This module employs Apache Spark, a powerful distributed processing system, to handle large datasets efficiently. Spark's capabilities in data cleaning, transformation, and integration ensure that the final datasets are both accurate and ready for analysis or application use.
- **Database:** For storage and management of processed data, the Database module utilizes

MongoDB. As a NoSQL database, MongoDB is well-suited for handling the flexible, unstructured data often encountered in web scraping projects, offering scalability and high performance for managing large volumes of data.

- **Web Application:** The final module is a web application that provides a user-friendly interface for interacting with processed data. Developed using Flask, a lightweight web framework, and integrated with the Gemini API, this application supports book search functionalities based on user questions and requests. It offers both keyword search and more advanced semantic search capabilities, enhancing the user experience by allowing for more intuitive and relevant search results.

Together, these modules form a cohesive system that automates the entire data lifecycle - from initial collection to final application - ensuring that the system operates with minimal manual intervention while maintaining high levels of accuracy, efficiency, and reliability.

2.1 Selenium

Selenium is an open-source automation framework widely recognized for its application in testing web applications, but it has also gained significant traction in the field of web scraping and web crawling due to its robust capabilities in automating browser interactions. At its core, Selenium facilitates the automation of web browsers, enabling developers to programmatically control browser actions in a way that closely mimics human interactions. This feature is particularly valuable in web crawling, where the ability to navigate complex, dynamic websites is crucial for extracting data that is not readily accessible through traditional methods.

In the context of web crawling, Selenium is employed to automate the process of accessing web pages, interacting with elements on the page, and extracting the desired data. Unlike traditional web scraping techniques that may struggle with dynamically loaded content, Selenium excels in environments where JavaScript-driven content needs to be rendered and interacted with before data extraction can occur. This makes Selenium an indispensable tool for crawling modern web applications that rely heavily on client-side scripting.

Key features of Selenium relevant to web crawling include:

- **Cross-Browser Automation:** Selenium supports automation across different browsers, including Chrome, Firefox, and Edge, ensuring compatibility with a wide range of web applications.

- **Dynamic Content Handling:** Selenium can interact with JavaScript and AJAX elements, allowing it to effectively handle and extract data from dynamically generated content.
- **Robust Element Interaction:** Selenium enables automated interaction with web elements, such as buttons, forms, and dropdowns, making it possible to simulate user actions required for navigating and extracting data from web pages.
- **Integration with Programming Languages:** Selenium is compatible with various programming languages, including Python, Java, and C#, offering flexibility in integrating it with existing web crawling frameworks and data processing pipelines.
- **Support for Headless Browsing:** Selenium can operate in a headless mode, meaning it can run without launching a graphical user interface (GUI), which is advantageous for large-scale web crawling tasks where efficiency and speed are paramount.
- **Efficient Page Load Management:** Selenium provides control over page load strategies, allowing crawlers to manage how and when to interact with a web page, thus optimizing the crawling process by reducing unnecessary waiting times.

Through these features, Selenium empowers developers to build sophisticated web crawlers capable of navigating and extracting data from complex, modern web applications, making it a critical tool in the arsenal of web scraping and data extraction technologies.

2.2 Airflow

Apache Airflow is an open-source platform designed to programmatically author, schedule, and monitor workflows. It allows you to define complex data pipelines and automate tasks, making it easier to manage and orchestrate data processing workflows. Some key features of Airflow:

- **Workflow Management:** Airflow allows you to define workflows as Directed Acyclic Graphs (DAGs), where each node represents a task and edges represent dependencies between tasks. This structure helps in visualizing and managing the sequence of tasks in a workflow.
- **Scalability:** Airflow is designed to handle workflows of varying complexity, from simple batch jobs to large-scale data processing pipelines. It supports horizontal scaling, allowing you to distribute tasks across multiple workers.

- **Extensibility:** Airflow provides a rich set of operators and hooks to interact with various data sources and systems, such as databases, cloud services, and APIs. You can also create custom operators and sensors to fit your specific needs.
- **Scheduling:** Airflow includes a powerful scheduler that triggers tasks based on a defined schedule or event. It supports cron-like scheduling, which allows you to run tasks at specific intervals or times.
- **Monitoring and Logging:** Airflow has an intuitive web-based user interface for monitoring workflows. You can view the status of each task, check logs for debugging, and manage task retries and dependencies.
- **Modularity:** The platform is modular, meaning you can integrate it with various systems and tools through its extensible architecture. This makes it suitable for orchestrating diverse workflows and processes.
- **Dynamic Pipelines:** Airflow allows for dynamic pipeline generation, enabling workflows to be defined programmatically. This means you can generate tasks and DAGs based on external parameters or conditions.

Overall, Airflow is a versatile tool for managing data workflows, making it easier to automate and orchestrate complex data processing tasks and pipelines. In this project, we use Apache Airflow to automate the crawling and saving of book data. Airflow's scheduling capabilities allow us to define and execute a series of tasks that crawl various sources for book information on a regular basis.

2.3 Spark

Apache Spark is an open-source, distributed computing system designed for processing large datasets quickly and efficiently. It provides an interface for programming entire clusters with implicit data parallelism and fault tolerance.



Figure 2: Spark Overview

Some key features of Spark:

- **Speed:** Spark processes data in memory, which makes it much faster than traditional disk-based processing systems like Hadoop. It can run programs up to 100 times faster in memory and 10 times faster on disk.
- **Ease of Use:** Spark supports multiple programming languages, including Python, Scala, Java, and R. It provides high-level APIs that make it easier for developers to write distributed applications.
- **General Purpose:** Spark can handle a variety of data processing tasks, including batch processing, stream processing, machine learning, and interactive querying. It has libraries like Spark SQL, Spark Streaming, MLib (for machine learning), and GraphX (for graph processing).
- **Unified Engine:** Spark can integrate with other big data tools and frameworks, such as Hadoop, Apache Hive, and Apache Kafka. It can also be deployed on a variety of cluster managers, including Hadoop YARN, Apache Mesos, Kubernetes, and its own standalone cluster manager.
- **Resilience:** Spark's RDD (Resilient Distributed Datasets) is the fundamental data structure that enables fault-tolerant, distributed processing of data. If a node fails, Spark can recompute lost data using the lineage of transformations applied to the original RDD.

Spark is commonly used in data engineering, data science, and machine learning tasks, especially when working with large-scale datasets. In this project, we utilize Apache Spark to clean and integrate book data from multiple sources. Spark's advanced distributed processing capabilities enable us to efficiently handle and process large volumes of data. The data cleaning process involves removing duplicates, correcting inconsistencies, and standardizing formats to ensure data

quality. Integration involves merging data from different sources into a unified dataset, which allows for a comprehensive view of the book information.

2.4 MongoDB

MongoDB is a popular, open-source NoSQL database known for its flexibility, scalability, and performance. Unlike traditional relational databases that use tables and rows, MongoDB stores data in a flexible, JSON-like format called BSON (Binary JSON). Some key features and aspects of MongoDB:

- **Document-Oriented Storage:** MongoDB stores data as documents in collections. Each document is a JSON-like object that can contain various data types, including nested arrays and other documents. This schema-less design allows for easy storage of complex and hierarchical data.
- **Scalability:** MongoDB is designed to scale horizontally by distributing data across multiple servers. It supports sharding, which means data can be split across different nodes, allowing the database to handle large amounts of data and high-throughput operations efficiently.
- **Flexible Schema:** MongoDB does not require a fixed schema for its documents. This flexibility allows you to store documents with different structures in the same collection and easily adapt to changes in your data model without requiring complex migrations.
- **High Performance:** MongoDB provides high performance for read and write operations through features like in-memory processing, indexing, and efficient querying. It supports various indexing options to optimize query performance.
- **Rich Query Language:** MongoDB offers a powerful query language that supports a wide range of query operations, including filtering, sorting, and aggregations. It also provides support for geospatial queries and text search.
- **Replication and High Availability:** MongoDB supports replication through replica sets, which are groups of MongoDB servers that maintain the same data. This ensures high availability and data redundancy, allowing automatic failover in case of server failures.
- **Aggregation Framework:** MongoDB includes an aggregation framework that allows you to perform complex data processing and transformation tasks, such as grouping, filtering, and aggregating data, directly within the database.

- **Ease of Use:** MongoDB has a user-friendly interface and provides various tools and libraries for different programming languages, making it easier to integrate with applications and manage data.

Overall, MongoDB is well-suited for applications requiring flexible data models, large-scale data storage, and high-performance operations, such as content management systems, real-time analytics, and Internet of Things (IoT) applications. In this project, we use MongoDB as our data warehouse to store and manage book data. MongoDB's flexible, document-oriented storage model allows us to efficiently handle diverse and complex data structures associated with book information.

2.5 Flask

Flask is a lightweight, open-source web framework for Python. It is designed to make it easy to build web applications and APIs with minimal setup and complexity. Here are some key features and aspects of Flask:

- **Microframework:** Flask is often referred to as a microframework because it provides the essential tools to build web applications without imposing a lot of overhead. It includes only the core features needed for web development, with the option to extend functionality using third-party libraries.
- **Simplicity and Flexibility:** Flask is known for its simplicity and flexibility. It provides the basic tools for routing, request handling, and templating, allowing developers to choose additional components and libraries based on their specific needs.
- **Routing:** Flask uses a routing system that maps URLs to functions or views. This allows developers to define URL patterns and handle HTTP requests in a straightforward manner.
- **Templating:** Flask uses Jinja2 as its default templating engine. Jinja2 allows you to generate dynamic HTML pages by embedding Python-like expressions and control structures within HTML templates.
- **Development Server:** Flask comes with a built-in development server that allows for easy testing and debugging of applications during development. This server is not intended for production use, but it simplifies the development process.
- **Extensible:** While Flask provides the basics, it is highly extensible. You can integrate

various extensions to add functionality such as database integration (e.g., SQLAlchemy), authentication, form handling, and more.

- **RESTful APIs:** Flask is well-suited for building RESTful APIs. Its simplicity and flexibility make it easy to create endpoints, handle requests, and return responses in a structured format such as JSON.
- **Configuration:** Flask provides a simple configuration system that allows you to manage application settings using configuration files or environment variables. This makes it easy to adjust settings for different environments (development, testing, production).
- **Community and Documentation:** Flask has a strong community and extensive documentation. This support makes it easier for developers to find resources, get help, and learn best practices.

Overall, Flask is a popular choice for developers who want to build web applications or APIs quickly and with minimal boilerplate code, while retaining the flexibility to choose additional components and libraries as needed. In this project, we use Apache Spark as our backend to process book search queries. Spark's powerful data processing capabilities enable us to efficiently handle large volumes of book data and execute complex search operations in real-time.

III. Data Crawler

In developing a solution to crawl data from the website, the approach taken involved two key stages: retrieving URLs of individual book entries from a given page and then extracting detailed information about each book. The process was designed to handle the dynamic and sometimes unpredictable nature of web scraping, with specific attention to potential challenges and how they were addressed.

- The first stage involved constructing a function to retrieve URLs of book entries on a specific page. The primary difficulty encountered here was ensuring that the scraper could reliably identify and extract the correct links from the HTML content. This required careful analysis of the website's structure, particularly the use of CSS classes to identify the relevant HTML elements. To mitigate the risk of errors due to changes in the webpage's structure, the code was designed to include robust error handling, such as checking the HTTP response status and implementing fallback mechanisms if the expected elements were not found.
- In the second stage, the focus shifted to extracting detailed attributes of each book, such as the title, author, ISBN, publisher, and publication year. One of the primary challenges in this phase was handling the variability in how these details were presented on the website. For instance, the format of the ISBN and publication year required precise string manipulation, which was prone to errors if the webpage deviated from the expected format. To address this, the code was designed to be flexible, with conditional checks to handle cases where certain elements might be missing or formatted differently. This ensured that the extraction process remained reliable across different book entries.

Another significant challenge was the potential for missing or incomplete data, particularly in fields such as author names or publication years, which could affect the quality of the dataset. To counter this, the solution incorporated checks to validate the extracted data, ensuring that only complete and accurate information was stored.

Tech stack

- WonderBK Crawler: **Requests + BeautifulSoup**
- Amazon Crawler: **Selenium**
- Goodreads Crawler: **Selenium**

goodreads

The Big Books of Fall

Discover readers' most anticipated new releases

Search books

Q

Sign in

Join

Home

My Books

Browse

Community

Most popular books published in August 2024

Books most frequently added to Goodreads members' shelves, updated weekly

2024

August



#1

Daydream (Maple Hills, #3)

Hannah Grace

★ 4.28 1,822 ratings · 256k shelveings

Want to read

The third in the New York Times bestselling Maple Hills series follows fan favorite Henry and a bookish fellow student who come up with a plan to help them both overcome their



#2

By Any Other Name

Jodi Picoult

★ 4.16 1,368 ratings · 162k shelveings

Want to read

Two women, centuries apart—one of whom is the real author of Shakespeare's plays—are both forced to hide behind another name to make their voices heard.



#3

Born of Blood and Ash (Flesh and Fire, #4)

Jennifer L. Armentrout

★ 4.36 4,969 ratings · 18k shelveings

Want to read

From #1 New York Times bestselling author Jennifer L. Armentrout comes the thrilling conclusion to her beloved Flesh and Fire series...

Show more

New releases

Discussion

Listops

Must read

Popular group books

Popular shelves

goodreads

The Big Books of Fall

Discover readers' most anticipated new releases

Search books

Q

Sign in

Join

Home

My Books

Browse

Community



Maple Hills #3

Daydream

Hannah Grace

★ ★ ★ ★ ★ 4.28

2,037 ratings 256k shelveings

The third in the New York Times bestselling Maple Hills series follows fan favorite Henry and a bookish fellow student who come up with a plan to help them both overcome their respective challenges in a difficult year.

When his procrastination lands him in a difficult class with his least favorite professor, Henry Turner

Show more

Genres **Romance** Sports Romance Sports Contemporary Hockey Contemporary Romance Fiction ...more

448 pages, Paperback

Expected publication August 27, 2024

Original title Daydream

Series Maple Hills (#3)

Characters Henry Turner, Helle Jacobs

This edition

Format 448 pages, Paperback

Expected publication August 27, 2024 by Alma Books

ISBN 9781668026250 (ISBN10: 1668026252)

ASIN B088026252

Language English

More editions

Kindle Edition Simon & Schuster UK

Kindle Edition Alma Books

Paperback Simon & Schuster

Paperback Editions B

Goodreads Website

IV. Data integration

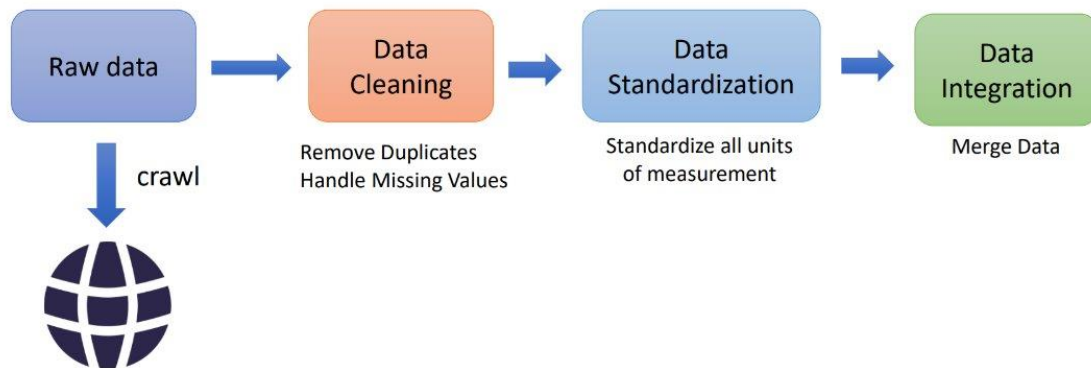


Figure 3: Data integration pipeline

Our overall data integration pipeline follows Figure 3 and consists of four steps:

Step 1: Crawl Book Sources

- Objective: Collect book data from various sources.
- Actions: Use web crawlers to extract information from book sources.

Step 2: Clean Data

- Objective: Prepare raw data for integration by addressing issues.
- Actions: Handle missing values, correct errors, and normalize data formats to ensure consistency.

Step 3: Data Normalization

- Objective: Standardize data to facilitate accurate integration.
- Actions: Apply normalization techniques to align data formats and values, making it easier to compare and merge information from different sources.

Step 4: Merge Data

- Objective: Combine data from multiple sources into a unified dataset.

- Actions: Integrate the cleaned and normalized data, resolving any conflicts and ensuring a coherent, comprehensive dataset.

First, we have three sources after crawling: Amazon, Goodreads, and WonderBK.

Data source

These sources were selected due to their diverse range of book types and the extensive information they provide. After crawling these sources, we obtain various pieces of information:

	ISBN	Book-Title	Book-Author	Year-Of-Publication	Publisher	Image-URL-S
0	0195153448	Classical Mythology	Mark P. O. Morford	2002	Oxford University Press	http://images.amazon.com/images/P/0195153448.0...
1	0002005018	Clara Callan	Richard Bruce Wright	2001	HarperFlamingo Canada	http://images.amazon.com/images/P/0002005018.0...
2	0060973129	Decision in Normandy	Carlo D'Este	1991	HarperPerennial	http://images.amazon.com/images/P/0060973129.0...

Amazon

	bookID	title	authors	average_rating	isbn	isbn13	language_code	num_pages	ratings_count	text_reviews_cou
0	1	Harry Potter and the Half-Blood Prince (Harry ...	J.K. Rowling/Mary GrandPré	4.57	0439785960	9780439785969	eng	652	2095690	275
1	2	Harry Potter and the Order of the Phoenix (Har...	J.K. Rowling/Mary GrandPré	4.49	0439358078	9780439358071	eng	870	2153167	292

Goodreads

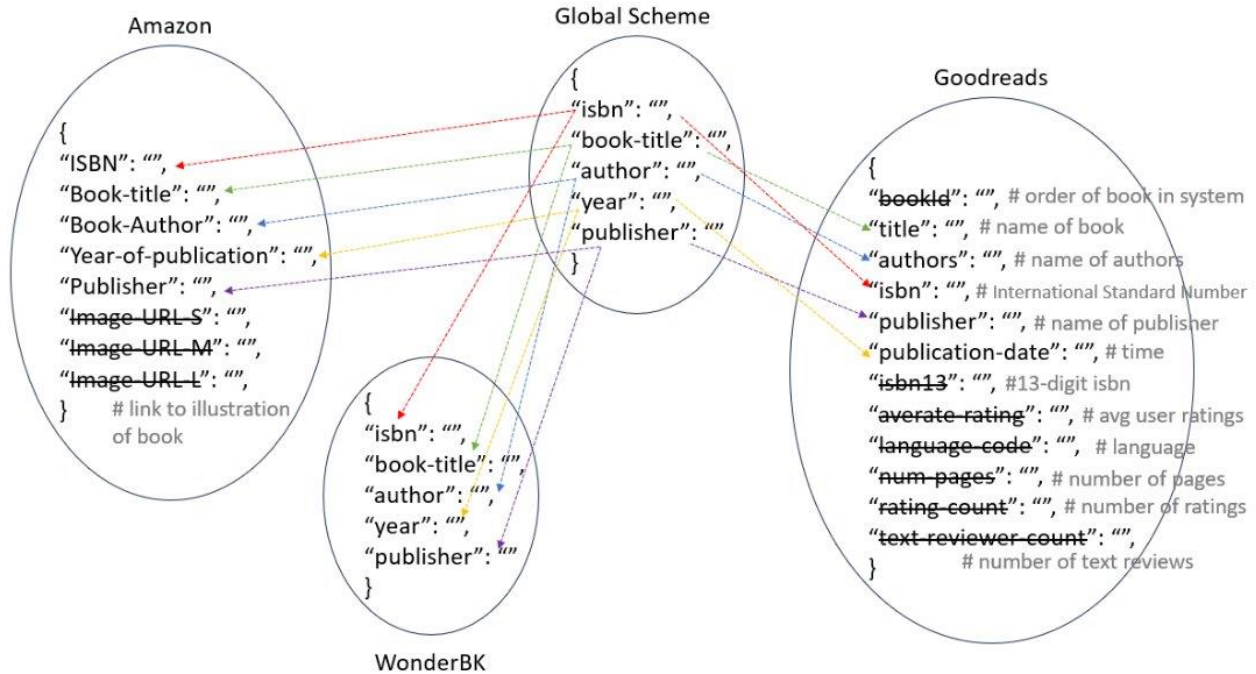
	Unnamed: 0	isbn	book_title	publisher	year	author
0	0	038524407X	Goat Brothers	Doubleday	1993	Colton, Larry
1	1	0399125876	The Missing Person	Putnam Pub Group	1981	Grumbach, Doris
2	2	089480488X	Don't Eat Your Heart Out Cookbook	Workman Pub Co	1983	Piscatella, Joseph C.
3	3	096274509X	Amy Spangler's Breastfeeding : A Parent's Guide	Amy Spangler	1997	Spangler, Amy
4	4	096582070X	The Foundation of Leadership: Enduring Princip...	Excalibur Press	1997	Short, Bo

WonderBK

Then we normalize and clean data:

- Date of Publication: Dates from the three sources are in different formats (e.g., Amazon: "2002", WonderBK: "Sunday, March 1, 1981", Goodreads: "9/16/2006"). Normalize these dates by extracting and retaining only the year.
- Author Names: Author names may include prefixes and postfixes (e.g., "By [Author]"). Remove all prefixes and postfixes, keeping only the author's name (e.g., "By [Author]" becomes "Author").
- Titles: Some titles contain special characters (e.g., *, \$, ...). Use regular expressions (regex) to remove all special characters, retaining only alphanumeric characters.

Next, we create a mediated schema and map the schemas extracted from various sources to this unified schema.



Schema Extraction

Finally, we merge data from multiple sources. A key challenge in this process is identifying and removing duplicates and merging records identical across different sources.

To merge two records, the first approach we use is using Levenshtein distance to assess similarity. If the character error rate between two titles is less than 0.2 and the character error rate between two sorted author names is also less than 0.2, we consider the records to be the same and merge them accordingly.

An example following figure:



$$CER = \frac{S + D + I}{N}$$

However, when merging records, we encounter several challenges:

- Inconsistent Author Formats:

- Some records list a single author, while others provide a list of authors.
- The order of authors in the list may differ ([A, B, C] vs. [C, B, A, D]).
- Author names may be formatted differently, such as first name first or last name first (e.g., "J.R.R. Tolkien" vs. "Tolkien, J.R.R.").
- Example Cases:
 - Case 1:
 - Title: *Ortho's All About Roses (Ortho's All About Gardening)*
 - Record 1: "By Ortho (EDT) and Cairns, Thomas"
 - Record 2: "Thomas Cairns"
 - **Issue:** The records refer to the same book, but the author's information is presented differently, making it challenging to merge.
 - Case 2:
 - Title: *The Internet for Dummies (4th Edition)*
 - Record 1: "By Levine, John R., Young, Margaret Levine, and Baroudi, Carol", 1997
 - Record 2: "John R. Levine", 1997
 - **Issue:** Although the title and publication year match, the author details are fragmented and formatted differently, complicating the merging process.

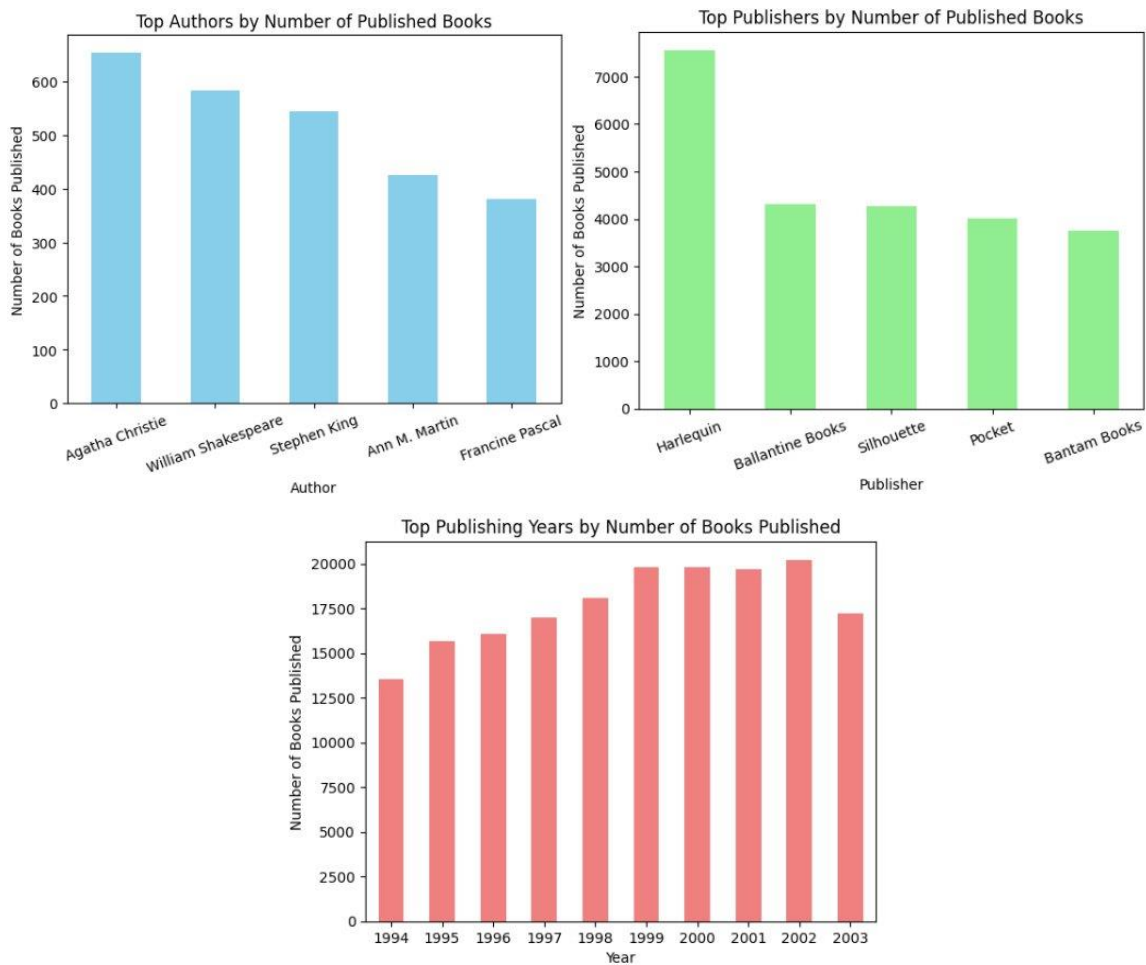
So, we use different method to handle merging and deduplication, we rely on the unique ISBN for each book, following these rules:

- Rule 1: Each book is uniquely identified by its ISBN, so we merge records based on this identifier.
- Rule 2: If the ISBN from Goodreads or WonderBK matches an ISBN from Amazon, we prioritize the information from Amazon.

After applying these rules and matching records from all three sources, our system detected approximately 30,000 duplicate records. This reduced the total number of records from 380,000 to 350,000.

V. Visualization and application

After finalizing the dataset, we visualized the data to uncover insights and trends. We then created a demo book search web application using Flask and HTML. This web app allows users to search for books, view detailed information, and interact with the integrated dataset, demonstrating the practical utility of the cleaned and merged data.



When visualizing the data, we observed several key insights:

- **Top Authors:** The most famous authors are prominently represented, showing a concentration of books by a few well-known names.
- **Author Distribution:** There is a substantial gap among authors, with some having significantly more books than others.

- **Publisher Analysis:** The number of books published by Harlequin is almost double that of other publishers. Meanwhile, the number of books from publishers outside the top publisher is quite similar.
- **Publication Trends:**
 - There is an increasing trend in the number of books published from 1994 to 1999.
 - This trend flattens between 1999 and 2002.
 - There is a significant drop in the number of books published after 2002, likely due to the rise in electronic books.

Finally, the demo book search web application includes comprehensive search capabilities. Users can find book information using criteria such as title, author, publisher, or year. Additionally, the application features a chatbot that allows users to query book information through natural language. The chatbot utilizes Gemini to extract relevant fields from user input, and the queries are processed using Spark to deliver accurate results efficiently.

Book Search

All Book

Query:

Query anything you want :D

Search

Success! Results ready!

#	Book	Author	Publisher	Year
1	Bilbo, Le Hobbit	J. R. R. Tolkien	Pocket (FR)	2002
2	Das Silmarrion	J. R. R. Tolkien	Distribooks	2001
3	Der kleine Hobbit.	John Ronald Reuel Tolkien	Dtv	1999
4	Der kleine Hobbit. Sonderausgabe.	John Ronald Reuel Tolkien	Dtv	2001

VI. Conclusion

In this project, we have designed a comprehensive book data integration system that leverages various tools and technologies to handle data scraping and management effectively:

- **Data Crawling:** We collected data from open sources to build a robust dataset.
- **Data Processing:** We utilized Spark for efficient data processing and integration.
- **Data Visualization:** We visualized the data to gain insights and understand trends.
- **Web Search Application:** We created a web search interface using Flask, allowing users to find book information through a user-friendly platform.

Future Plans:

- **Integrate More Data Sources:** We aim to expand our system by incorporating additional data sources for a more comprehensive dataset.
- **Build a Content-Based Recommendation System:** We plan to develop a recommendation system based on book content to enhance user experience and provide personalized suggestions.